

JAVA — METHODS / FUNCTIONS

Day 07 • Student Classroom / Projector Notes

From repeated code → reusable tasks

A method is a named block of code designed to perform a specific task.

Core Flow: Problem → Task → Method → Call → Execute → Return / Output

1. WHY DO WE NEED METHODS?

- Reduce repeated code.
- Give a task a meaningful name.
- Make programs easier to read and test.
- Break a large problem into smaller tasks.
- Reuse the same logic whenever required.

```
// Without a method  
System.out.println("Hello");  
System.out.println("Hello");  
System.out.println("Hello");
```

```
// With a method  
greet();  
greet();  
greet();
```

2. METHOD ANATOMY

- **static** — modifier used in these class examples.
- **void** — method does not return a value.
- **methodName** — meaningful name of the task.
- **parameters** — input received by the method.
- **body** — statements that perform the task.
- **return** — sends a value back when a method has a return type.

```
static void greet() {  
    System.out.println("Hello!");  
}
```

3. DECLARATION vs CALL

- **Declaration/definition:** describes what the method does.
- **Call:** tells Java to execute the method.
- A method does not execute merely because it is declared.

```
static void greet() {  
    System.out.println("Hello!");  
}
```

```
public static void main(String[] args) {  
    greet();  
    greet();  
}
```

4. VOID METHODS

- Use **void** when the method does not need to send a value back.
- A void method can perform an action such as printing or displaying.

```
static void greet(String name) {  
    System.out.println("Hello, " + name);  
}
```

```
static void printLine() {  
    System.out.println("-----");  
}
```

5. PARAMETERS & ARGUMENTS

- **Parameter:** variable written in the method definition.
- **Argument:** actual value supplied during the call.
- The number and types of arguments must match the parameters.

```
static void greet(String name) {  
    System.out.println("Hello, " + name);  
}
```

```
greet("Ravi");  
// name = parameter  
// "Ravi" = argument
```

```
static int add(int a, int b) {  
    return a + b;  
}
```

```
int ans = add(10, 20);
```

6. RETURN VALUES

- Use a return type when a method must give a value back.
- The returned value can be stored, printed, compared or used in another expression.
- **Printing a value is not the same as returning a value.**

```
static int square(int n) {  
    return n * n;  
}
```

```
int x = square(5);  
System.out.println(x);
```

```
if (square(5) > 20) {  
    System.out.println("Large");  
}
```

7. PRINT vs RETURN

- **print:** sends information to the screen.
- **return:** sends a value back to the caller.
- Use return when another part of the program needs to use the result.

```
static void squarePrint(int n) {  
    System.out.println(n * n);  
}
```

```
static int square(int n) {  
    return n * n;  
}
```

8. METHOD COMPOSITION

- One method can call another method.
- Break a large task into smaller reusable tasks.
- Reuse existing logic instead of copying it.

```
static int square(int n) {  
    return n * n;  
}
```

```
static int sumOfSquares(int a, int b) {  
    return square(a) + square(b);  
}
```

```
System.out.println(sumOfSquares(3, 4));  
// 25
```

9. LOCAL VARIABLES & SCOPE

- A local variable is declared inside a method or block.
- It can be used only within its accessible scope.
- Parameters are local to the method that receives them.

```
static int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

```
// sum cannot be directly used here
```

10. STATIC

- **static** is a modifier.
- In these beginner examples, static methods can be called from static main without creating an object.
- static does not mean “method”.

```
static void greet() {  
    System.out.println("Hello");  
}
```

```
public static void main(String[] args) {  
    greet();  
}
```

11. METHOD OVERLOADING

- Same method name can be used for different parameter lists.
- The parameter list must differ by number and/or parameter types.
- **Changing only the return type is not enough for overloading.**

```
static int add(int a, int b) {  
    return a + b;  
}
```

```
static int add(int a, int b, int c) {  
    return a + b + c;  
}
```

```
System.out.println(add(1, 2));  
System.out.println(add(1, 2, 3));
```

12. OVERLOADING — QUICK CHECK

- `add(int, int) + add(double, double) → valid.`
- `add(int, int) + add(int, int) → not valid.`
- Different number of parameters → valid.
- Changing only return type → not valid.

13. PASS-BY-VALUE

- Java passes arguments by value.
- For primitive variables, the method receives a copy of the value.
- Changing the parameter does not change the original primitive variable.

```
static void change(int x) {  
    x = 100;  
}
```

```
int a = 10;  
change(a);
```

```
System.out.println(a);  
// 10
```

14. METHOD-FIRST PROBLEM SOLVING

- 1. Identify the task.
- 2. Give it a meaningful method name.
- 3. Identify required inputs.
- 4. Decide the output.
- 5. Choose parameter types.
- 6. Choose return type or void.
- 7. Write the method signature.
- 8. Write the logic.
- 9. Call the method.
- 10. Test normal and edge cases.

15. COMMON REUSABLE METHODS

- `isEven(n)` → checks even/odd.
- `max(a,b)` → returns larger value.
- `maxOfThree(a,b,c)` → returns largest value.
- `square(n)` / `cube(n)` → calculations.
- `countDigits(n)` → number of digits.
- `sumDigits(n)` → sum of digits.
- `reverseNumber(n)` → reverses digits.
- `isPalindrome(n)` → palindrome check.
- `isPrime(n)` → prime check.
- `factorial(n)` / `power(a,b)` → mathematical tasks.

16. COUNT DIGITS

- Count the digits of an integer.
- Special case: 0 has one digit.
- Negative numbers can be handled using absolute value.

```
static int countDigits(int n) {  
    n = Math.abs(n);  
  
    if (n == 0)  
        return 1;  
  
    int count = 0;  
  
    while (n > 0) {  
        count++;  
        n /= 10;  
    }  
  
    return count;  
}
```

17. REVERSE NUMBER

- Last digit $\rightarrow n \% 10$
- Remove last digit $\rightarrow n /= 10$
- Build reverse $\rightarrow rev = rev * 10 + digit$

```
static int reverseNumber(int n) {  
    int sign = n < 0 ? -1 : 1;  
    n = Math.abs(n);  
  
    int rev = 0;  
  
    while (n > 0) {  
        int digit = n % 10;  
        rev = rev * 10 + digit;  
        n /= 10;  
    }  
  
    return sign * rev;  
}
```

18. PALINDROME

- A number is a palindrome if it reads the same forward and backward.
- 121 → palindrome
- 123 → not palindrome
- One approach: compare the original number with its reverse.

```
static boolean isPalindrome(int n) {  
    return n == reverseNumber(n);  
}
```

19. PRIME NUMBER

- Prime numbers have exactly two positive factors.
- 0 and 1 are not prime.
- Test divisors while $i \times i \leq n$.

```
static boolean isPrime(int n) {  
    if (n < 2)  
        return false;  
  
    for (int i = 2; i * i <= n; i++) {  
        if (n % i == 0)  
            return false;  
    }  
  
    return true;  
}
```

20. DSA CONNECTION

- Methods divide DSA problems into small reusable tasks.
- Searching, sorting, number problems and data-structure operations can be separated into methods.
- Method composition prepares you for larger programs and recursion.
- Think about correctness, edge cases and time complexity.

21. BUG HUNT — FIND THE ERRORS

- Find the conceptual/compiler problems before running the code.

```
static int square(int n) {  
    System.out.println(n * n);  
    // Problem?  
}
```

```
static void add(int a, int b) {  
    return a + b;  
    // Problem?  
}
```

```
static int max(int a, int b) {  
    if (a > b)  
        return a;  
    // Problem?  
}
```

22. COMMON MISTAKES

- Confusing parameter and argument.
- Forgetting to call the method.
- Wrong number/type of arguments.
- Returning a value from void.
- Missing return in a non-void method.
- Thinking print and return are the same.
- Trying to overload by changing only return type.
- Using a local variable outside its scope.
- Copy-pasting logic instead of reusing methods.
- Forgetting edge cases such as 0, 1 and negative numbers.

23. CLASSROOM PRACTICE — METHOD DESIGN

- For each problem write: Method Name → Parameters → Return Type → Logic → Call.
- 1. Method that returns whether a number is even.
- 2. Method that returns maximum of two numbers.
- 3. Method that returns maximum of three numbers.
- 4. Method that returns square of a number.
- 5. Method that returns sum of digits.
- 6. Method that returns reverse of a number.
- 7. Method that checks palindrome.
- 8. Method that checks prime.

24. FINAL CHALLENGE — NUMBER ANALYZER

- Input an integer n.
- Using separate methods, display: digit count, digit sum, reverse, palindrome, prime and even/odd.
- Do not put all logic in main.
- Use meaningful method names.
- Test normal values and edge cases.

main

```
countDigits(n)
sumDigits(n)
reverseNumber(n)
isPalindrome(n)
isPrime(n)
isEven(n)
```

25. EXIT TICKET

- 1. What is a parameter?
- 2. What is an argument?
- 3. Difference between print and return?
- 4. Can a void method return a value?
- 5. Can return type alone overload a method?
- 6. What does Java pass to a primitive parameter?
- 7. Why divide a large problem into methods?
- 8. What should you decide before writing a method?

METHOD MINDSET

One method = one clear responsibility.

Name the task → define the input → decide the output → write the method → call it → test it → reuse it.